

Dynamic Memory Allocation



Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot



Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}
Programming



Dynamic Memory Allocation (DMA)

- ▶ If memory is allocated at runtime (during execution of program) then it is called dynamic memory.
- ▶ It allocates memory from **heap** (*heap*: it is an empty area in memory)
- ▶ Memory can be accessed only through a pointer.

When DMA is needed?

- ▶ It is used when number of variables are not known in advance or **large** in size.
- ▶ Memory can be allocated at any time and can be released at any time during runtime.

malloc() function

- ▶ **malloc ()** is used to allocate a fixed amount of memory during the execution of a program.
- ▶ **malloc ()** allocates **size_in_bytes** of memory from heap, if the allocation succeeds, a pointer to the block of memory is returned else **NULL** is returned.
- ▶ Allocated memory space may not be contiguous.
- ▶ Each block contains a **size**, a pointer to the next block, and the space itself.
- ▶ The blocks are kept in ascending order of storage address, and the last block points to the first.
- ▶ The memory is not initialized.

Syntax	Description
<pre>ptr_var = (cast_type *) malloc (size_in_bytes);</pre>	This statement returns a pointer to size_in_bytes of uninitialized storage, or NULL if the request cannot be satisfied. Example: <code>fp = (int *)malloc(sizeof(int) *20);</code>

Write a C program to allocate memory using malloc.

Program

```
1 #include <stdio.h>
2 void main()
3 {
4     int *fp; //fp is a pointer variable
5     fp = (int *)malloc(sizeof(int)); //returns a pointer to int size storage
6     *fp = 25; //store 25 in the address pointed by fp
7     printf("%d", *fp); //print the value of fp, i.e. 25
8     free(fp); //free up the space pointed to by fp
9 }
```

Output

25

calloc() function

- ▶ `calloc()` is used to allocate a block of memory during the execution of a program
- ▶ `calloc()` allocates a region of memory to hold `no_of_blocks` of `size_of_block` each, if the allocation succeeds then a pointer to the block of memory is returned else `NULL` is returned.
- ▶ The memory is initialized to `ZERO`.

Syntax	Description
<pre>ptr_var = (cast_type *) calloc (no_of_blocks, size_of_block);</pre>	This statement returns a pointer to <code>no_of_blocks</code> of size <code>size_of_block</code>, it returns <code>NULL</code> if the request cannot be satisfied. Example: <pre>int n = 20; fp = (int *)calloc(n, sizeof(int));</pre>

Write a C program to allocate memory using calloc.

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      int i, n; //i, n are integer variables
5      int *fp; //fp is a pointer variable
6      printf("Enter how many numbers: ");
7      scanf("%d", &n);
8      fp = (int *)calloc(n, sizeof(int)); //calloc returns a pointer to n blocks
9      for(i = 0; i < n; i++) //loop through until all the blocks are read
10     {
11         scanf("%d",fp); //read and store into location where fp points
12         fp++; //increment the pointer variable
13     }
14     free(fp); //frees the space pointed to by fp
}
```

realloc() function

- ▶ **realloc()** changes the size of the object pointed to by pointer `fp` to specified size.
- ▶ The contents will be unchanged up to the minimum of the old and new sizes.
- ▶ If the new size is larger, the new space will be uninitialized.
- ▶ **realloc()** returns a pointer to the new space, or **NULL** if the request cannot be satisfied, in which case `*fp` is unchanged.

Syntax	Description
<pre>ptr_var = (cast_type *) realloc (void *fp, size_t);</pre>	This statement returns a pointer to new space, or NULL if the request cannot be satisfied. Example: <code>fp = (int *)realloc(fp, sizeof(int)*20);</code>

Write a C program to allocate memory using realloc.

Program

```
1 #include <stdio.h>
2 void main()
3 {
4     int *fp; //fp is a file pointer
5     fp = (int *)malloc(sizeof(int)); //malloc returns a pointer to int size storage
6     *fp = 25; //store 25 in the address pointed by fp
7     fp =(int *)realloc(fp, 2*sizeof(int)); //returns a pointer to new space
8     printf("%d", *fp); //print the value of fp
9     free(fp); //free up the space pointed to by fp
}
```

Output

25

free() function

- ▶ free deallocates the space pointed to by fp.
- ▶ It does nothing if fp is **NULL**.
- ▶ fp must be a pointer to space previously allocated by **calloc**, **malloc** or **realloc**.

Syntax	Description
<code>void free(void *);</code>	This statement free up the memory not needed anymore. Example: <code>free(fp);</code>

Write a C program to sort numbers using malloc

Program

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  void main()
4  {
5      int i,j,t,n;
6      int *p;
7      printf("Enter value of n: ");
8      scanf("%d", &n);
9      p=(int *) malloc(n * sizeof(int));
10     printf("Enter values\n");
11     for(i=0; i<n; i++)
12         scanf("%d", &p[i]);
13     for(i=0; i<n; i++)
14     {
15         for(j= i+1; j<n; j++)
16         {
```

Program (cont.)

```
17             if(p[i] > p[j])
18             {
19                 t = p[i];
20                 p[i] = p[j];
21                 p[j] = t;
22             }
23         }
24     }
25     printf("Ascending order\n");
26     for(i=0; i<n; i++)
27         printf("%d\n", p[i]);
28     free(p);
29 }
```

Write a C program to find square of numbers using calloc

Program

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  void main()
4  {
5      int i,n;
6      int *p;
7      printf("Enter value of n: ");
8      scanf("%d",&n);
9      p=(int*)calloc(n,sizeof(int));
10     printf("Enter values\n");
11     for(i=0;i<n;i++)
12         scanf("%d",&p[i]);
13     for(i=0;i<n;i++)
14         printf("Square of %d = %d\n", p[i],
15 p[i] * p[i]);
16     free(p);
17 }
```

Output

```
Enter value of n: 3
Enter values
3
2
5
Square of 3 = 9
Square of 2 = 4
Square of 5 = 25
```

Write a C program to add/remove item from a list using realloc

Program

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  void main()
4  {
5      int i, n1, n2;
6      int *fp;
7      printf("Enter size of list: ");
8      scanf("%d", &n1);
9      fp=(int *) malloc (n1 * sizeof(int));
10
11     printf("Enter %d numbers\n", n1);
12     for(i = 0; i < n1; i++)
13         scanf("%d", &fp[i]);
14
15     printf("The numbers in the list are\n");
16     for(i = 0; i < n1; i++)
17         printf("%d\n", fp[i]);
```

Program (cont.)

```
18
19     printf("Enter new size of list: ");
20     scanf("%d", &n2);
21
22     fp = realloc(fp, n2 * sizeof(int));
23     if(n2 > n1)
24     {
25         printf("Enter %d numbers\n", n2 - n1);
26         for(i = n1; i < n2; i++)
27             scanf("%d", &fp[i]);
28     }
29     printf("The numbers in the list are\n");
30     for(i = 0; i < n2; i++)
31         printf("%d\n", fp[i]);
32 }
```

Practice Programs

- 1) Write a C program to calculate sum of n numbers entered by user.
- 2) Write a C program to input and print text using DMA
- 3) Write a C program to read and print student details using structure and DMA



Thank you

